

Agentic AI — Interview Questions & Prep Sheet

Covers: LLM Fundamentals → AI Agents → Agentic AI → MCP Servers → GitHub Copilot (Ask/Plan/Agent modes, Custom Agents) → Claude Code (Skills & Subagents)

Section 1: LLM Fundamentals (Foundation)

- 1. What is a Large Language Model (LLM)? How does it generate a response?** An LLM is a neural network trained on massive text corpora to predict the next token in a sequence. Given a prompt, it generates text on learned probability distributions —The base LLM itself predicts tokens and does not independently access tools, persistent memory, or external systems. An application or agent runtime can provide these capabilities around the model.
 - 2. What are the core limitations of a plain LLM (without any agent framework around it)?** No persistent memory across sessions, no access to real-time/external data, no ability to take actions (run code, call APIs, browse), and it can hallucinate confidently. It only produces text — it doesn't *do* anything.
 - 3. What is the difference between a chatbot and an LLM-based application?** A chatbot is one narrow use case (conversational Q&A) built on an LLM. An LLM application can extend far beyond chat — code generation, agents, RAG pipelines, orchestration systems, etc.
-
-

Section 2: AI Agents — What & Why

4. What is an AI agent? How is it different from a plain LLM call?

An AI agent is an LLM combined with the ability to reason, plan, use tools, take actions, observe results, and iterate toward a goal — autonomously, with minimal step-by-step human instruction. A plain LLM call just returns text for a single prompt; an agent operates in a loop (perceive → plan → act → observe → repeat).

5. What are tokens, and how do they affect cost, latency, and context-window usage?

Tokens are the units processed by an LLM. Larger prompts, conversation histories, and tool results consume more context, increase cost, and may increase latency.

6. What is "context window" and why does it matter for agentic systems?

It's the maximum amount of text (tokens) the model can process at once, including prompt, history, and tool outputs. Agent design (subagents, skills) exists partly to manage context — offloading work so the main session's window doesn't get flooded with irrelevant detail.

7. What are the key components of an AI agent architecture?

LLM (reasoning core), tools/functions it can call, memory (short-term/context and sometimes long-term/persistent) and an environment it acts in (filesystem, APIs, browser, etc.).

8. What does "tool use" or "function calling" mean in the context of AI agents?

The model's ability to recognize when it needs external capability (search, code execution, database query) and call a defined function/tool with structured parameters, then use the returned result to continue reasoning.

9. Give a real-world example of an AI agent vs. a non-agentic LLM use case.

Non-agentic: asking ChatGPT "summarize this paragraph." Agentic: "go find our Q3 sales numbers, compare them to Q2, and draft a report" — where the system decides which tools to use, fetches data, computes, and produces output

without being told each step.

Section 3: Agentic AI (The Bigger Picture)

10. **What is "Agentic AI"? How is it different from a single AI agent?** Agentic AI refers to systems where one or more agents autonomously plan, decide, and coordinate — often with multiple specialized agents working together (orchestration) rather than one agent doing everything. It's the broader paradigm; "an agent" can be one building block within it.
 11. **What is agent orchestration, and why is it needed?** Orchestration is when a main/coordinator agent decomposes a complex goal into subtasks and delegates them to specialized subagents, then combines their results. It's needed because a single agent handling everything in one context leads to context overload, slower iteration, and less reliable results on complex, multi-part tasks.
 12. **What distinguishes "agentic" behavior from just following instructions step by step?** Agentic behavior means the system decides *how* to break down and solve a goal — which tools/skills/subagents to invoke and in what order — versus a human explicitly scripting every step. The hallmark is autonomous planning and delegation.
 13. **What are common risks or challenges with agentic AI systems?** *Model answer:* Compounding errors across multi-step chains, tool/permission misuse, cost and latency from multiple agent calls, difficulty debugging emergent behavior, and the need for guardrails (scoped tool access, validation steps) to keep autonomy safe.
-

Section 4: MCP Servers (Model Context

Protocol)

14. **What is an MCP server, and what problem does it solve?** MCP (Model Context Protocol) is a standard protocol that lets an AI model/agent connect to external tools and data sources (databases, APIs, file systems, SaaS apps) in a consistent way. It solves the "every integration needs custom glue code" problem by giving agents a uniform way to discover and call external capabilities.
15. **Give an example of when you'd use an MCP server in an agent workflow.** Connecting an agent to Playwright MCP (for Browser Automation), Jira (to pull tickets), or Google Drive (to read docs) — the agent calls the MCP server's exposed tools instead of you writing custom API code for each.
16. **How do MCP servers relate to "tools" that an agent can call?** MCP servers expose a set of tools/resources that get registered as callable tools within the agent's toolset — so from the model's perspective, an MCP-provided tool looks just like any other tool it can invoke.
17. **What security risks arise when connecting an agent to an MCP server?**

Risks include malicious tool descriptions, excessive permissions, data leakage, prompt injection, unauthorized actions, and compromised servers. MCP guidance explicitly treats tool-related metadata as untrusted unless it comes from a trusted server

Section 5: GitHub Copilot — Modes & Custom Agents

18. **What is the difference between "Ask," "Plan," and "Agent" mode in GitHub Copilot?** *Model answer:*

- **Ask mode:** conversational Q&A — you ask, it answers/suggests, but doesn't take autonomous action on your codebase.
 - **Plan/Planning mode:** helps research and propose a design/approach before execution — useful for scoping a solution before code changes happen.
 - **Agent mode:** autonomously decides which files to edit, runs terminal commands, and iterates on errors with minimal manual intervention — it actually executes the work.
19. **When would you use Planning mode instead of jumping straight into Agent mode?** *Model answer:* When the task is ambiguous, architecturally significant, or needs a reviewed approach first — Planning mode lets you validate the "how" before autonomous execution changes real files.
20. **What is a "custom agent" in GitHub Copilot, and how do you define one?** *Model answer:* A custom agent is a lightweight, scoped agent definition with its own name, system prompt, and restricted toolset (and optional MCP servers), created so Copilot can delegate matching requests to it automatically as a sub-agent running in isolated context.
21. **How does Copilot decide which custom agent to delegate a task to?** The Copilot runtime matches the incoming request against each custom agent's defined expertise/description and automatically routes to the best-fit agent — similar to how Claude Code auto-invokes matching skills.
-

Section 6: Claude Code — Skills System & Subagents

22. **What is a "Skill" in Claude Code? What does a SKILL.md file contain?** A Skill is reusable procedural knowledge — a SKILL.md file (name, description/trigger condition) plus instructions Claude follows when the skill is relevant. It's knowledge Claude loads into

context, not a separate running process.

23. **What is a Subagent in Claude Code, and how is it different from a Skill?** *Model answer:* A Subagent is a separate, isolated Claude instance with its own context window that the main agent delegates a specific task to; it runs independently and returns only a summary. A Skill is passive know-how that gets loaded/invoked — by the main agent or by a subagent — it doesn't run its own isolated session. Skills = knowledge; Subagents = isolated execution.
24. **If you give Claude Code one high-level prompt and it automatically invokes multiple skill files/subagents to solve the problem — what concept does that demonstrate?** That's agentic AI in action: you gave a goal, not step-by-step instructions, and the system autonomously planned, decided which skills/subagents to invoke, and combined the results — the defining trait of agentic behavior vs. simple instruction-following.
25. **Why would you choose a Subagent/SubSkills over just adding more instructions to the main session's system prompt?** *Model answer:* To avoid flooding the main context with logs/search results/file contents that aren't needed long-term, to get parallelism on independent subtasks, and to keep permission/tool scope tightly restricted per task.
-

Section 7: Scenario / Applied Questions

26. **You need an agent system that reviews a pull request against your team's coding standards, using data from GitHub. Design it — which pieces (Skill, Subagent, MCP) would you use and why?** MCP server (GitHub MCP) for PR diff/comment access; a Skill encoding the team's review checklist (security, naming, test coverage); a Subagent ("pr-reviewer") with read-only tools that calls the MCP server, loads the skill for methodology, and returns a summary to the main session.
-